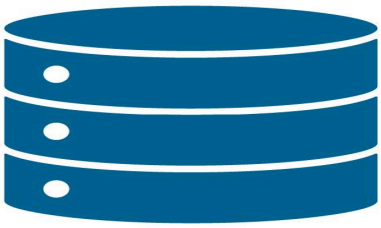
Unidad 1

Almacenamiento de la información



FICHA TÉCNICA DE LA UNIDAD

Unidad Didáctica	Unidad 1: Almacenamiento de la información
Resultados de Aprendizaje	**RA1: Almacena información en bases de datos utilizando sistemas de gestión de bases de datos.**
Criterios de Evaluación	CE1.a, CE1.b, CE1.c, CE1.d, CE1.e, CE1.f.
Tiempo Estimado	12 horas (distribuidas en sesiones de teoría, codificación y talleres de diseño)

ÍNDICE DE CONTENIDOS

1 ALMACENAMIENTO DE LA INFORMACIÓN

2 SISTEMAS DE ARCHIVOS

- 2.1 ORGANIZACIÓN PRIMARIA DE ARCHIVOS
- 2.2 MÉTODOS DE ACCESO

3 SISTEMAS DE BASES DE DATOS

3.1 Conceptos

3.2 ARQUITECTURA DE SISTEMAS DE BASES DE DATOS

3.2 MODELOS DE DATOS

3.3 TIPOS DE MODELOS

4 SISTEMAS GESTORES DE BASES DE DATOS

4.1 DEFINICIÓN Y OBJETIVOS

4.2 FUNCIONES DEL SISTEMA GESTOR DE BASE DE DATOS

4.3 COMPONENTES DE UN SGBD

4.4 Lenguajes de datos

4.5 USUARIOS DE LOS SGBD

4.6 TIPOS DE SGBD

4.7 SISTEMAS GESTORES DE BD COMERCIALES Y LIBRES

5 BD CENTRALIZADAS Y DISTRIBUIDAS

5.1 TÉCNICAS DE FRAGMENTACIÓN, REPLICACIÓN Y DISTRIBUCIÓN

Importante: los apartados marcados con 📖, son de lectura obligatoria aunque no se expliquen durante las clases teóricas.

1 ALMACENAMIENTO DE LA INFORMACIÓN

Todas las aplicaciones informáticas trabajan en última instancia con datos o información que deben ser almacenados en un medio físico, como discos duros, memorias flash o DVD. Estos medios forman una jerarquía que distingue entre tres niveles de almacenamiento: primario, secundario e intermedio.

Almacenamiento primario

Se refiere a aquellos medios sobre los que la CPU del ordenador puede acceder directamente y, por tanto, más rápidamente. Son la memoria principal o memoria RAM y las memorias caché de primer y segundo nivel, más pequeñas pero más rápidas.

Almacenamiento secundario

El almacenamiento secundario de más amplio uso es el disco, aunque las cintas se usan sobre todo para copias de seguridad por su estabilidad, capacidad y durabilidad.

Se refiere a dispositivos más lentos, pero de mayor capacidad, como los discos ópticos y magnéticos o las cintas. Para acceder a los datos la CPU debe copiarlos previamente en el almacenamiento primario.

La transferencia de información entre memoria y disco tiene lugar en unidades de bloque. Cuando se produce una orden de lectura se copia uno o varios bloques en el llamado buffer de la memoria (área reservada de la memoria principal), si se necesita efectuar una escritura se copia el bloque correspondiente al bloque del disco.

Los discos son dispositivos de acceso aleatorio ya que podemos acceder a cualquier bloque de información solamente conociendo su dirección física en el disco, sin necesidad de recorrerlos todos.

Así mismo, las cintas son dispositivos de acceso secuencial, dado que los bloques se almacenan de manera contigua y para acceder a uno de ellos necesitamos leer todos los anteriores.

Almacenamiento intermedio

Cuando se necesita transferir varios bloques de disco a memoria principal y se conocen todas las direcciones de bloque es posible reservar varias áreas de almacenamiento intermedio o buffers dentro de la memoria principal para agilizar la transferencia. De este modo, mientras la CPU procesa datos de un buffer puede leer o escribir en otro. El uso de este tipo de almacenamiento es muy común en sistemas de bases de datos.

2 SISTEMAS DE ARCHIVOS

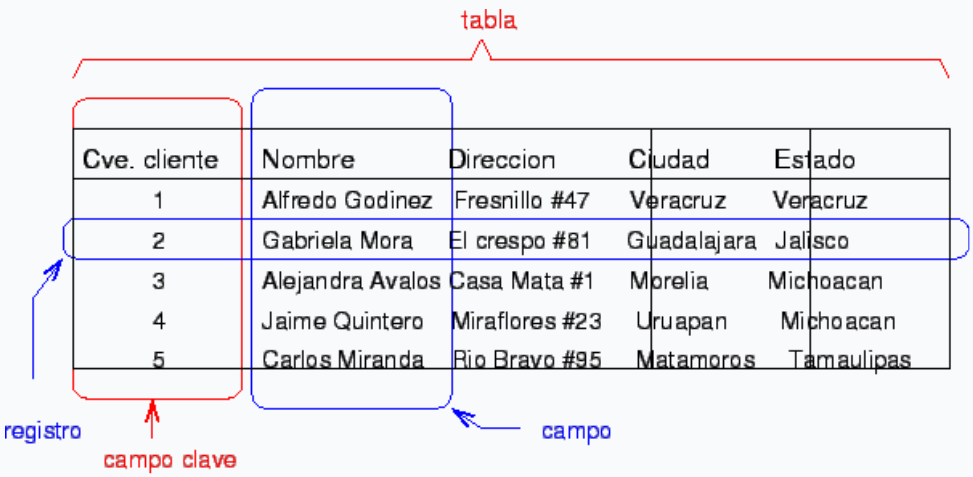
En esta sección estudiaremos someramente los conceptos relacionados con archivos tanto en lo relativo a su contenido como a su organización lógica y física.

Tradicionalmente, los ficheros se han clasificado de muchas formas, según su contenido (texto o binario), según su organización de la información (secuencial, directa, indexada) o según su utilidad (maestros, históricos, movimientos).

Registros

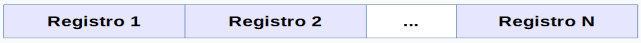
Normalmente la información se almacena en forma de **registros**, que son colecciones de valores o elementos de información relacionados, cada uno de los cuales corresponde a un campo del registro. Por ejemplo, un registro de alumno incluiría **campos** como el nombre, fecha de nacimiento o teléfono, cuyos valores para cada alumno forman cada registro. A su vez, cada campo tiene un **tipo de dato** que especifica el tipo de valores que puede tomar. Estos tipos suelen corresponderse con los lenguajes de programación. Así, en nuestro ejemplo tendríamos que para el nombre del alumno usamos un tipo carácter o char, para la fecha de nacimiento un tipo de fecha y para la edad un tipo numérico entero o int. Además, cada campo tiene un tamaño determinado en bytes que puede ser fijo o variable según los requisitos de la aplicación.

La unión de estos campos y sus tipos determina el tipo o formato del **registro**.



Archivos

Podemos definir un **fichero** informático como un conjunto de registros, grabados sobre un soporte que pueda ser leído por el ordenador.



(Imagen de creación propia)

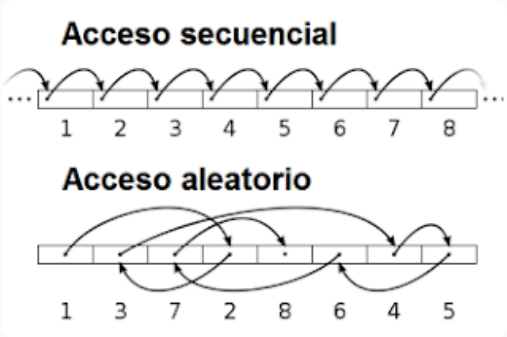
Estos registros pueden tener longitud fija si todos los registros son iguales en tamaño, o variable si los registros son de distinto tipo o si, aun siendo iguales en formato tienen campos de tamaño variable u opcionales (campos que no necesariamente tienen un valor para cada registro, por ejemplo, teléfono en el tipo de registro de alumno podría ser un campo opcional).

Los archivos de registros de longitud fija son más fáciles de manipular por parte de los programas, sin embargo desperdician espacio en disco. Por el contrario, para el caso de longitud variable los programas son más complejos ya que los registros no ocupan posiciones fijas, sino que dependen del valor de cada campo.

Los ficheros son importantes porque son la unidad básica de información utilizada por cualquier programa, incluidos los sistemas gestores de bases de datos. Todos los datos son, en última instancia gestionados mediante ficheros mediante cuatro operaciones básicas: **consulta o lectura, inserción, modificación y borrado**. Cualquier operación más compleja (como búsqueda, ordenación, etc.) es combinación de dos o más operaciones básicas.

2.1 ORGANIZACIÓN PRIMARIA DE ARCHIVOS

El término **organización de ficheros** se aplica a la forma en que se colocan los datos contenidos en los registros de cada fichero sobre el soporte informático (disco, cinta, etc.)



durante su grabación.

Existen dos formas básicas de organización de ficheros: secuencial y relativa. En la organización secuencial los registros se van grabando unos a continuación de los otros, en el orden que se van dando de alta, mientras que en la organización relativa los registros se graban en las posiciones que les corresponda según el valor que guarden en el campo denominado clave, que permite identificar el registro dentro del fichero.

Organización secuencial

Es el tipo más básico de organización. **Los registros se colocan secuencialmente uno a continuación del otro y los registros nuevos se añaden al final del fichero**.

La inserción es eficiente al realizarse al final del fichero, pero la búsqueda es costosa por ser lineal y bloque a bloque. El borrado es ineficiente; genera huecos que no se reutilizan automáticamente, requiriendo reorganizaciones periódicas para compactar los registros.

La modificación es compleja al exigir búsqueda y reescritura. En registros de longitud variable, si el nuevo dato no cabe, debe tratarse como una nueva inserción al final. Existen variantes para mejorar estas prestaciones en soportes direccionables.

Organización secuencial indexada

Los registros con los datos s***e graban en un fichero secuencialmente***, pero ***se pueden recuperar con acceso directo gracias a la utilización de un fichero adicional, llamado índice***, que contiene información de la posición que ocupa cada registro en el fichero de datos.

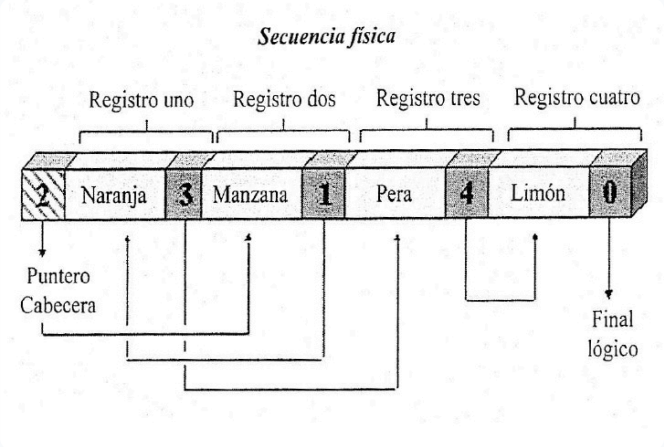


Llave	Ubicación
21	340
23	85
39	255
59	170
72	415
87	0

Clave	Nombre	Dirección	Deuda
87	María García Barro	Zaragoza No. 36	1,890.40
23	Elizabeth Espíritu Ortega	Tercera calle No. 81	4,721.67
59	María Alcocer Solache	Timas No. 29	2,894.78
39	Lilia Morales Flores	Roca No. 14	834.78
21	Thelma León Marcial	Caritino Maldonado 27	3,165.23
72	Guadalupe Velez Bernal	Erendida No. 103	1,589.23

La organización secuencial encadenada

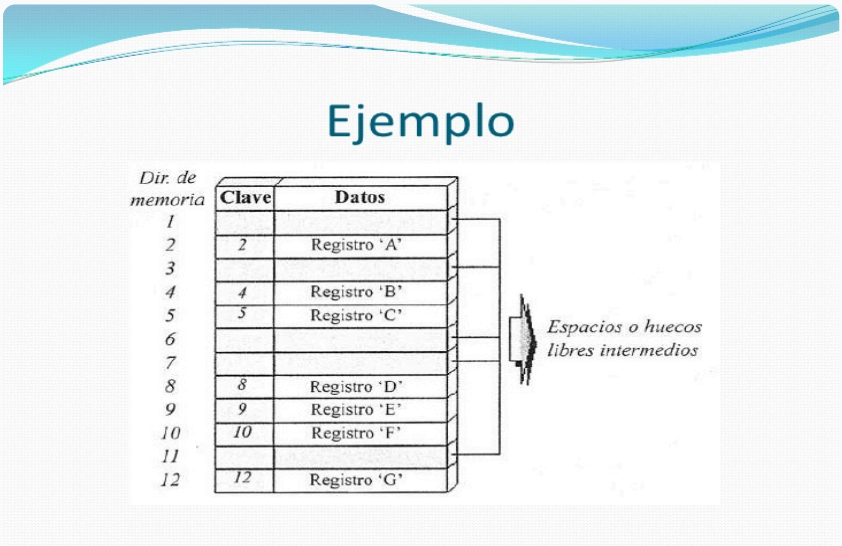
Permite tener los registros ***ordenados según un orden lógico diferente del orden físico*** en el que están grabados gracias a la utilización de unos campos adicionales llamados punteros.



Organización Relativa

En este tipo de archivos los registros ***se graban en orden según el valor de uno de sus campos llamado campo de ordenación***. Normalmente se usa un campo especial denominado ***campo clave***, cuyos valores son distintos para cada registro.

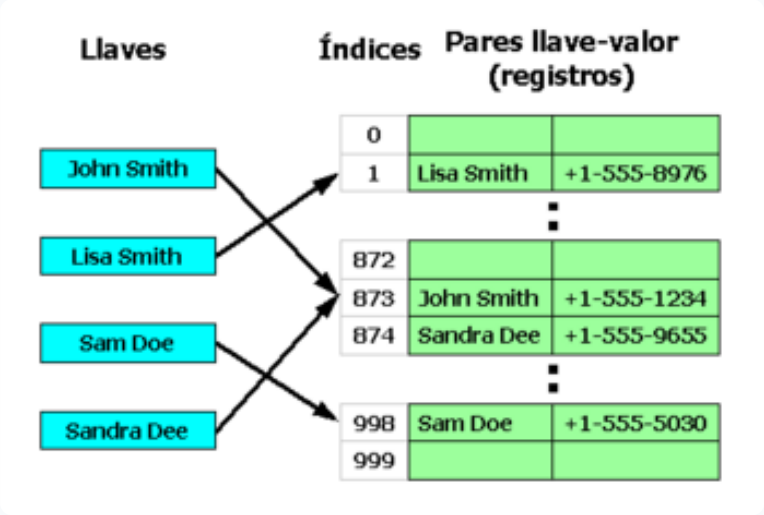
La organización relativa es ineficiente para accesos aleatorios o por campos distintos al de ordenación, requiriendo en esos casos archivos adicionales. Mantener el orden físico encarece las inserciones por el desplazamiento de registros, mientras que la modificación es compleja si afecta al campo clave o varía el tamaño del registro. La eliminación es más sencilla mediante marcas y reorganizaciones periódicas.



Dispersión

En este sistema, se aplica una ***función de aleatorización*** sobre un ***campo de dispersión*** para ***obtener la dirección del bloque de disco*** donde se guardará el registro. Estas técnicas de hashing agilizan las búsquedas individuales basadas en dicho campo, que suele ser la clave.

Las técnicas de dispersión o hashing se utilizan para acelerar el acceso a los registros cuando se busca un único registro según el llamado campo de dispersión. Normalmente este campo suele ser el campo clave.



Existen numerosas funciones de dispersión y su eficacia radica en que distribuyan los registros lo más posible minimizando el número de colisiones (producida cuando dos valores distintos del campo de dispersión producen el mismo valor de dispersión). Para estas situaciones se deben implementar medidas de corrección, como usar una segunda función, buscar la siguiente posición vacía dentro del bloque o usar técnicas de encadenamiento de registros.

2.2 MÉTODOS DE ACCESO

El método de acceso es el procedimiento para localizar registros en un fichero. Para optimizar esta costosa operación se emplean índices: estructuras que vinculan valores de un campo (normalmente la clave) con su dirección de memoria.

Similares a un índice analítico, estos listados ordenados facilitan búsquedas binarias rápidas al usar un campo de indización y apuntadores a bloques. Al ser el archivo de índices reducido, la eficiencia aumenta drásticamente.

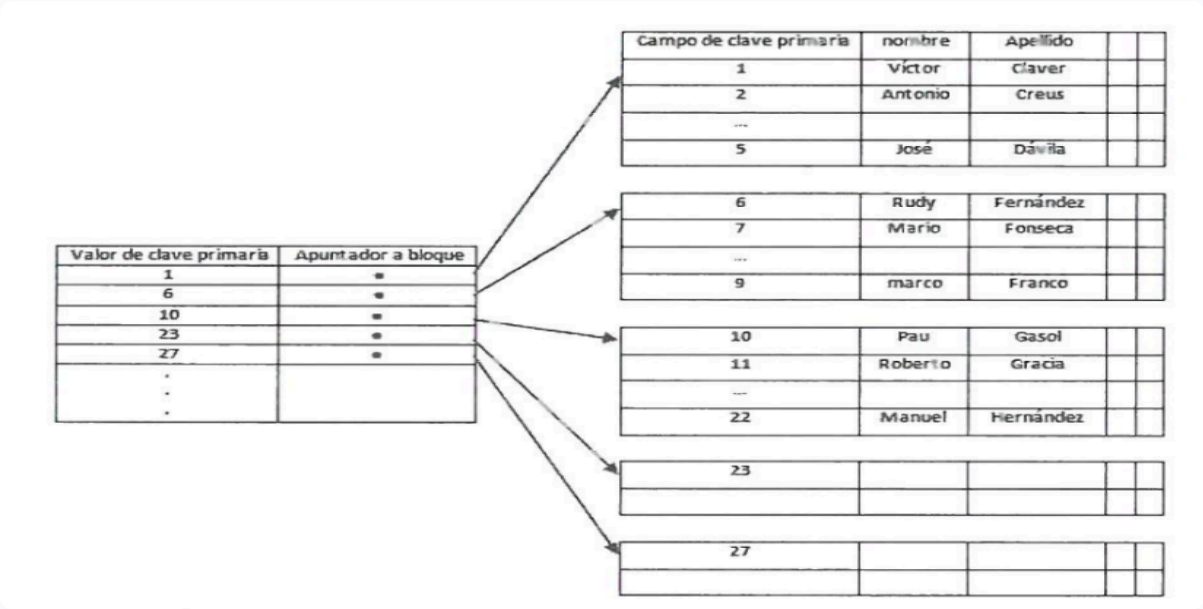
Existen varios tipos según la organización física:

- **Índices primarios:** sobre el campo clave de ordenación en ficheros ordenados físicamente.
- **Índice de agrupamiento:** usado cuando el campo de ordenación permite valores repetidos (no es clave).
- **Índice secundario:** cuando el campo de indización es distinto al de ordenación.

Índices primarios

Un índice primario es un fichero ordenado de registros de longitud fija con dos campos: la clave de ordenamiento del archivo de datos y un puntero al bloque de disco. Cada entrada vincula un valor clave con un bloque, apuntando así a un grupo de registros.

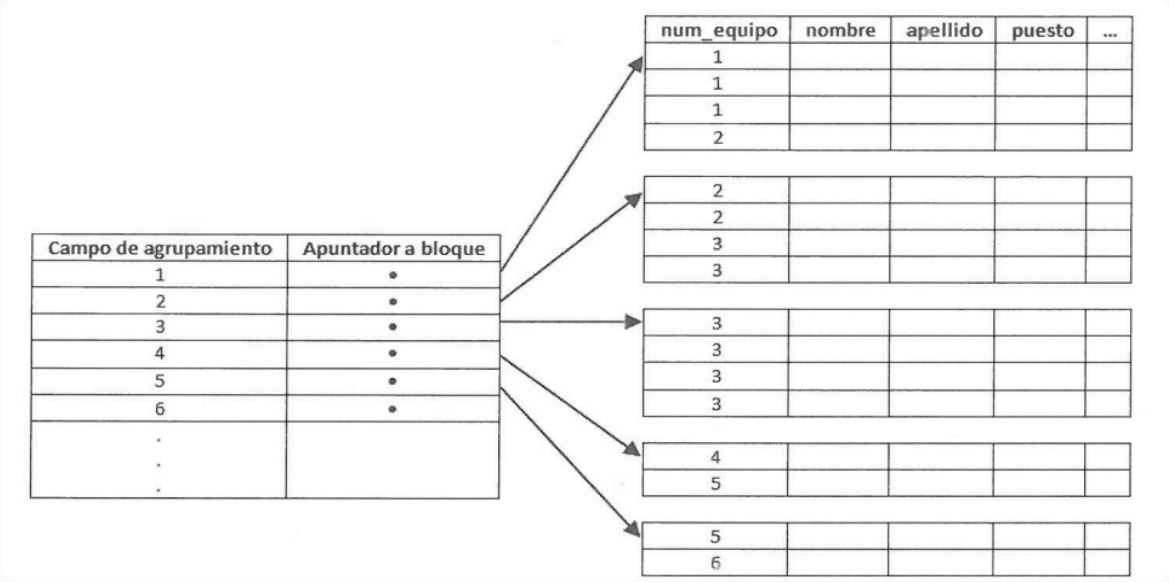
Por ejemplo, en un fichero de jugadores de baloncesto indizado por su ID secuencial, el índice solo registra el primer identificador de cada bloque.



Estos índices son no densos porque cada entrada se asocia a varios registros, a diferencia de los índices densos (relación uno a uno). Al contener sólo dos campos y una entrada por bloque, son mucho más pequeños que el fichero de datos original.

Índices de agrupamiento 📖

Cuando un archivo se ordena mediante un campo no clave (con valores repetidos), denominado campo de agrupamiento, se crea un índice de agrupamiento para agilizar la localización de registros. Este índice consiste en un fichero con el valor del campo y un puntero al primer bloque de datos que lo contiene.



Se suele asignar una entrada por cada valor distinto y reservar bloques específicos para facilitar inserciones, enlazando bloques adicionales mediante punteros si el volumen de datos lo requiere.

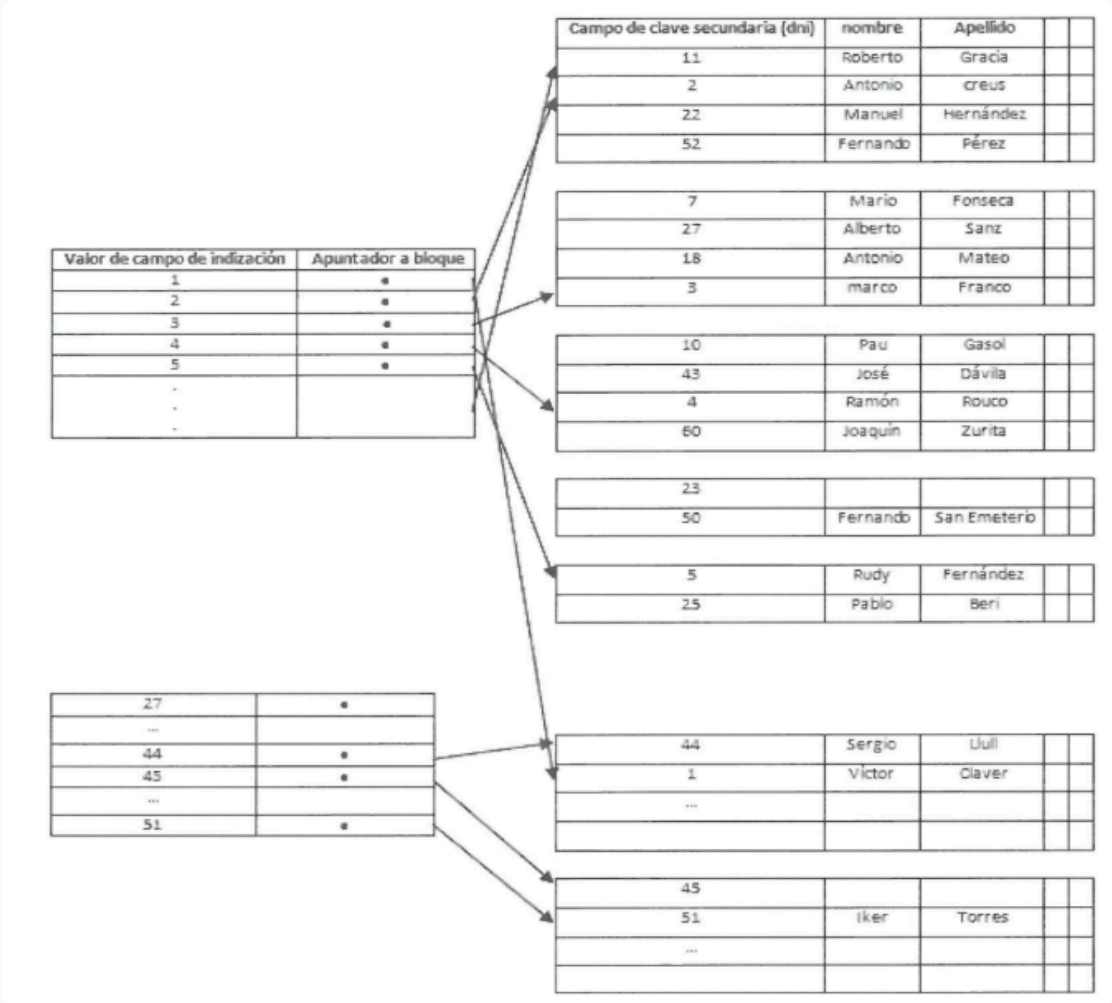
Un ejemplo práctico es la creación de este índice en un fichero de jugadores utilizando el número de equipo como criterio de ordenación.

En este caso el índice es sobre un campo que no es clave y se puede repetir, como es el número de equipo de los jugadores.

Hay una entrada de índice por cada equipo que apunta al bloque en el que se encuentra dicho equipo.

Índices secundarios

Estos archivos ordenados vinculan campos clave con apuntadores a bloques. Es posible definir múltiples índices sobre distintos campos de un mismo archivo de datos.



Existen índices sobre campos clave (índices densos con una entrada por registro) y sobre campos no clave. Un ejemplo común es el uso del DNI como clave secundaria para identificar jugadores.

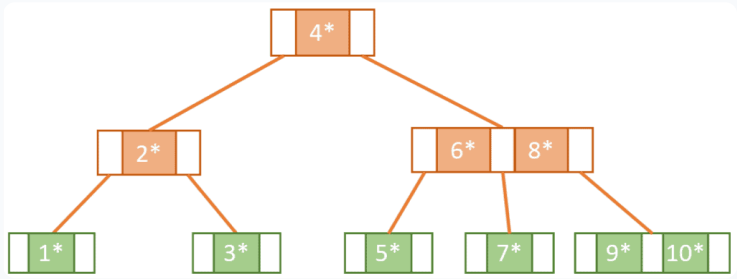
Aunque consumen más espacio por su volumen de entradas, optimizan las búsquedas; al estar ordenados, permiten aplicar búsquedas binarias mucho más rápidas que el acceso lineal

al archivo de datos original.

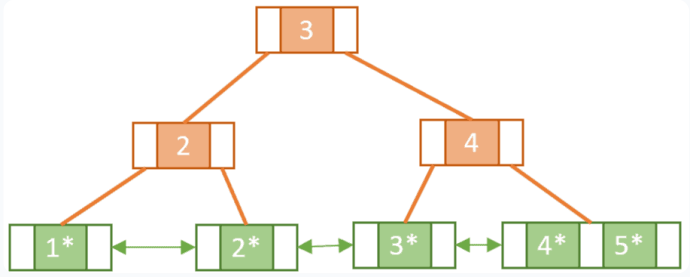
Otros tipos de índice 📖

Existen estructuras de índice más complejas.

Árboles B y B+ : Son estructuras de árbol muy utilizadas en programación y en índices de bases de datos. En este caso, un árbol es un fichero en el que hay unos nodos (padres) que apuntan a otros (hijos), los cuales a su vez tienen apuntadores a otros nodos y, así sucesivamente, con tantos nodos como entradas de índice requeridas.



Los nodos hijos no pueden apuntar a nodos padre, de modo que se forma una estructura con un nodo raíz sin padre cuyos nodos hijos son padres de otros hasta llegar a los nodos hoja, o nodos sin hijos.



Estos nodos almacenan apuntadores a nodos hijo, apuntadores a registros de datos y valores de campos de los registros de datos y la estructura que forman hace muy eficiente la búsqueda de datos.

Índices hash: Es posible crear estructuras de acceso de tipo índice usando técnicas de dispersión vistas en la sección anterior. En este caso, cada entrada de índice se organiza como un archivo de dispersión y contiene registros formados por el valor de dispersión y un apuntador al bloque de datos. Así, el acceso a una de ellas requiere aplicar la función hash correspondiente al valor buscado y obtener así el valor del apuntador.

ACTIVIDADES 1.1

1. Investigue y explique con sus palabras en qué consiste el método de búsqueda binaria en ficheros.

2. Realice un ejemplo de búsqueda secuencial y binaria en clase suponiendo que tiene que acceder a un valor dentro de un conjunto ordenado de valores. Compute y compare el número de lecturas en ambos procesos para varios valores de búsqueda.

3. Investiga las ventajas e inconvenientes respecto a la actualización de datos en ficheros con organización tipo hash.

4. Investigue la diferencia entre una estructura de índice tipo árbol B y B+.

5. Buscar diez extensiones que identifiquen formatos de ficheros binarios. Incluir una pequeña explicación del tipo de fichero (aplicación que los utiliza, utilidad,...).

3 SISTEMAS DE BASES DE DATOS

Inicialmente, cuando las primeras empresas y organizaciones empezaron a usar sistemas informáticos trabajaban con sistemas de ficheros. Cada equipo trabajaba con sus propios datos y programas y se encargaba de su mantenimiento y gestión. Al principio el sistema funcionó pero con el tiempo y, sobre todo, con el incremento de la cantidad de información así como de los usuarios que la manejaban surgieron problemas (**integridad** y **duplicidad** de información, **seguridad**, etc., que llevaron finalmente a la organización de la información mediante un sistema más ordenado y manejable basado en la centralización de la gestión y la organización de los datos en forma de bases de datos.

Pensar en una biblioteca donde existirían distintos archivadores (archivos o ficheros) donde los datos de los libros se organizaban por autor, por título, por la editorial. En cada archivador se repetiría información que ya está en otro.

No debemos olvidar que en última instancia todo se almacena en ficheros. La diferencia es que cuando usamos bases de datos no trabajamos directamente con ficheros, sino con estructuras de datos más fáciles de manejar.

Los principales problemas relacionados con el uso de ficheros como sistema de almacenamiento de información fueron los siguientes:

- **Separación y aislamiento de los datos:** Cuando los datos se separan en distintos ficheros es más complicado acceder a ellos, ya que el programador de aplicaciones debe sincronizar el procesamiento de los distintos ficheros implicados para asegurar que se extraen los datos correctos.

- **Duplicación de datos.** La redundancia de datos existente en los sistemas de ficheros hace que se desperdicie espacio de almacenamiento y, lo que es más importante, puede llevar a que se pierda la consistencia de los datos. Se produce una inconsistencia cuando copias de los mismos datos no coinciden.

- **Dependencia de datos.** Ya que la estructura física de los datos (la definición de los ficheros y de los registros) se encuentra codificada en los programas de aplicación, cualquier cambio en dicha estructura es difícil de realizar. El programador debe identificar todos los programas afectados por este cambio, modificarlos y volverlos a probar, lo que cuesta mucho tiempo y está sujeto a que se produzcan errores. A este problema, tan característico de los sistemas de ficheros, se le denomina también falta de independencia de datos lógica-física.

- **Formatos de ficheros incompatibles.** Ya que la estructura de los ficheros se define en los programas de aplicación, es completamente dependiente del lenguaje de programación. La incompatibilidad entre ficheros generados por

distintos lenguajes hace que los ficheros sean difíciles de procesar de modo conjunto.

- **Consultas fijas y proliferación de programas de aplicación.** Desde el punto de vista de los usuarios finales, los sistemas de ficheros fueron un gran avance comparados a los sistemas manuales. A consecuencia de esto, creció la necesidad de realizar distintos tipos de consultas de datos. Sin embargo, los sistemas de ficheros son muy dependientes del programador de aplicaciones: cualquier consulta o informe que se quiera realizar debe ser programado por él. En algunas organizaciones se conformaron con fijar el tipo de consultas e informes, siendo imposible realizar otro tipo de consultas que no se hubieran tenido en cuenta a la hora de escribir los programas de aplicación.
- **Control de concurrencia.** El acceso de varios clientes al mismo fichero genera inconsistencias, ya que un cliente puede consultar un fichero mientras otro lo está modificando.
- **Autorizaciones.** Los errores en los permisos de ficheros pueden hacer que un mismo cliente pueda modificar un dato en un fichero y no en otro en el que esté repetido.
- **Catálogo.** Resulta complicado saber dónde están los distintos datos. No existe un esquema general que muestre la organización de la información.

Para trabajar de un modo más efectivo, surgieron las bases de datos como modelo de organización de los datos y, con ellas, los sistemas de gestión de bases de datos (SGBD) como herramienta que permite la implementación y gestión de bases de datos.

3.1 Conceptos

Una **base de datos** es un conjunto de datos almacenados entre los que existen relaciones lógicas y ha sido diseñada para satisfacer los requerimientos de información de una empresa u organización.

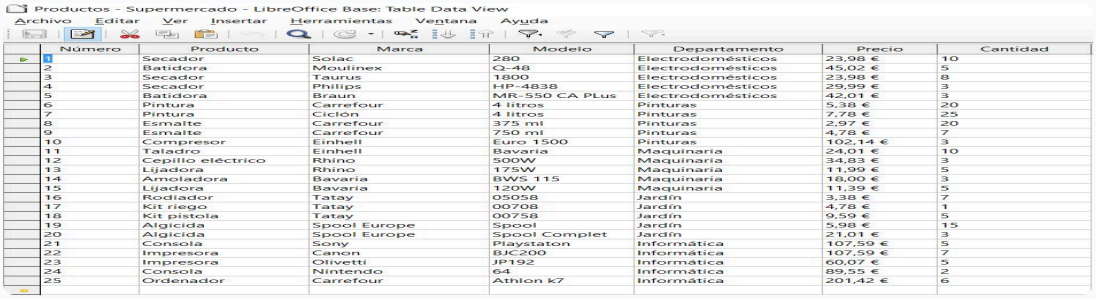
En lugar de almacenarse en ficheros desconectados y de manera redundante, los datos en una base de datos están centralizados y organizados, de forma que se minimice la redundancia y se facilite su gestión. La base de datos no pertenece a un equipo, se comparte por toda la organización. Además, la base de datos no solo contiene los datos de la organización, también almacena una descripción de dichos datos. Esta descripción es lo que se denomina **metadatos**, se almacena en el diccionario de datos o catálogo que, en muchos casos, se organiza en otra base de datos.

Conceptos

Uno de los grandes problemas al que se enfrentan los informáticos cuando comienzan su aprendizaje, es el gran número de términos desconocidos que debe asimilar, incluyendo el enorme número de sinónimos y siglas que se utilizan para nombrar la misma cosa. Tratando, a modo de resumen, de aclarar algunos de lo componentes que se pueden encontrar en una base de datos, se definen los siguientes conceptos:

- **Dato**: El dato es un trozo de información concreta sobre algún concepto o suceso. Por ejemplo, 1996 es un número que representa el año de

nacimiento de una persona. Los datos se caracterizan por pertenecer a un tipo.



Productos - Supermercado - LibreOffice Base: Table Data View

Archivo Editar Ver Insertar Herramientas Ventana Ayuda

Número	Producto	Marca	Modelo	Departamento	Precio	Cantidad
1	Secador	Solac	280	Electrodomésticos	23,98 €	10
2	Batidora	Moulinex	Q-48	Electrodomésticos	45,02 €	5
3	Secador	Taurus	1800	Electrodomésticos	23,98 €	8
4	Secador	Philips	HP-4838	Electrodomésticos	29,99 €	3
5	Batidora	Braun	MR-550 CA PLus	Electrodomésticos	42,01 €	3
6	Pintura	Carrefour	4 litros	Pinturas	5,38 €	20
7	Pintura	Ciclón	4 litros	Pinturas	7,78 €	25
8	Esmalte	Carrefour	375 ml	Pinturas	2,97 €	20
9	Esmalte	Carrefour	750 ml	Pinturas	4,78 €	7
10	Compresor	Einhell	Euro 1500	Pinturas	102,14 €	3
11	Taladro	Einhell	Bavaria	Maquinaria	24,01 €	10
12	Cepillo eléctrico	Rhino	500W	Maquinaria	34,83 €	3
13	Lijadora	Rhino	175W	Maquinaria	11,99 €	5
14	Amoladora	Bavaria	BWS 115	Maquinaria	18,00 €	3
15	Lijadora	Bavaria	120W	Maquinaria	11,39 €	5
16	Rodiador	Tatay	05058	Jardín	3,38 €	7
17	Kit riego	Tatay	00708	Jardín	4,78 €	1
18	Kit pistola	Tatay	00758	Jardín	9,59 €	5
19	Algicida	Spool Europe	Spool	Jardín	5,98 €	15
20	Algicida	Spool Europe	Spool Complet	Jardín	21,01 €	3
21	Consola	Sony	Playstaton	Informática	107,59 €	5
22	Impresora	Canon	BJC200	Informática	107,59 €	7
23	Impresora	Olivetti	JP192	Informática	60,07 €	5
24	Consola	Nintendo	64	Informática	89,55 €	2
25	Ordenador	Carrefour	Athlon k7	Informática	201,42 €	6

- **Tipo de Dato**: El tipo de dato indica la naturaleza del campo. Así, se puede tener datos numéricos, que son aquellos con los que se pueden realizar cálculos aritméticos (sumas, restas, multiplicaciones...) y los datos alfanuméricos, que son los que contienen caracteres alfabéticos y dígitos numéricos. Estos datos alfanuméricos y numéricos se pueden combinar para obtener tipos de datos más elaborados. Por ejemplo, una Fecha contiene tres datos numéricos, el día, el mes y el año.

- **Campo**: Un campo es un identificador para toda una familia de datos. Cada campo pertenece a un tipo de datos. Por ejemplo, el campo "FechaNacimiento" representa las fechas de nacimiento de las personas que hay en la tabla. Este campo pertenece al tipo de dato Fecha. Al campo también se le llama columna.

- **Registro**: Es una recolección de datos referentes a un mismo concepto o suceso. Por ejemplo, los datos de una persona pueden ser su NIF, año de nacimiento, su nombre, su dirección, etc. A los registros también se les llama tuplas o filas.

- **Campo Clave**: Es un campo especial que identifica de forma única a cada registro. Así, el NIF es único para cada persona, por tanto es campo clave.

- **Tabla**: Es un conjunto de registros bajo un mismo nombre que representa el conjunto de todos ellos. Por ejemplo, todos los clientes de una base de datos se almacenan en una tabla cuyo nombre es Clientes.

- **Consulta**: Es una instrucción para hacer peticiones a una base de datos. Puede ser una búsqueda simple de un registro específico o una solicitud para seleccionar todos los registros que satisfagan un conjunto de criterios. Aunque en castellano, consulta tiene un significado de extracción de información, en inglés query, una consulta es una petición, por tanto, además de las consultas de búsqueda de información, hay consultas (peticiones) de eliminación o inserción de registros, de actualización de registros, cuya ejecución altera los valores de los mismos.

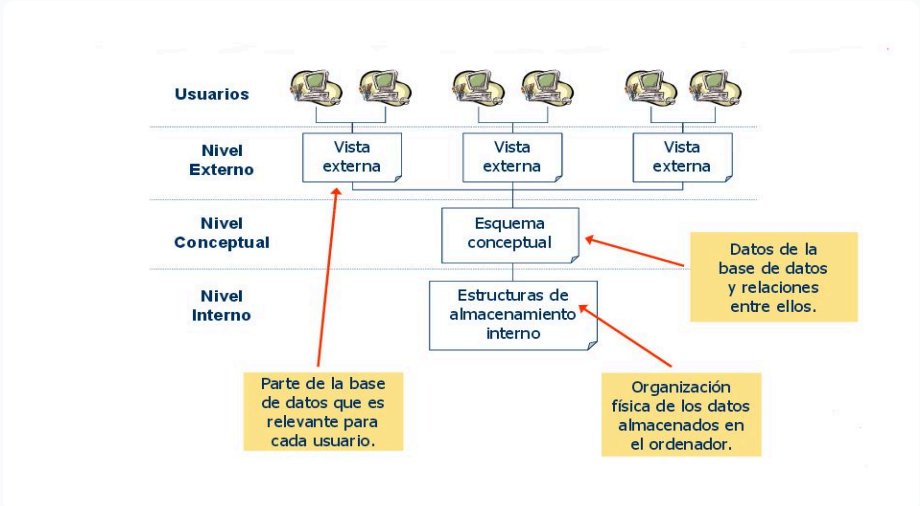
- **Índice**: Es una estructura que almacena los campos clave de una tabla, organizándose para hacer más fácil encontrar y ordenar los registros de esa tabla. El índice tiene un funcionamiento similar al índice de un libro, guardando parejas de elementos: el elemento que se desea indexar y su posición en la base de datos.

- **Vista**: Es una transformación que se hace a una o más tablas para obtener una nueva tabla. Esta nueva tabla es una tabla virtual, es decir, no está almacenada en los dispositivos de almacenamiento del ordenador, aunque sí se almacena su definición.
- **Informe**: Es un listado ordenado de los campos y registros seleccionados en un formato fácil de leer. Generalmente se usan como peticiones expresas de un tipo de información por parte de un usuario. Por ejemplo, un informe de las facturas impagadas del mes de enero ordenado por nombre de cliente.
- **Guiones: o scripts.** Son un conjunto de instrucciones, que ejecutadas de forma ordenada, realizan operaciones avanzadas de mantenimiento de los datos almacenados en la base de datos.
- **Procedimientos**: Son un tipo especial de script que está almacenado en la base de datos y que forma parte de su esquema.

3.2 ARQUITECTURA DE SISTEMAS DE BASES DE DATOS

En 1975, el comité ANSI-SPARC (American National Standard Institute - Standards Planning and Requirements Committee) propuso un estándar para la creación de sistemas de bases de datos basado en una arquitectura de tres niveles. El objetivo de la arquitectura de tres niveles es el de separar en niveles de abstracción el esquema de una base de datos. Son tres formas distintas de ver o representar una misma base de datos.

- **Nivel interno.** Se describe la estructura física de la base de datos mediante un esquema interno. Este esquema describe todos los detalles para el almacenamiento de la base de datos, así como los métodos de acceso. Se habla de ficheros, discos, directorios, etc.
- **Nivel global o conceptual.** Se describe la estructura de toda la base de datos para una comunidad de usuarios (todos los de una empresa u organización) mediante un esquema conceptual. Este esquema oculta los detalles de las estructuras de almacenamiento y se concentra en describir entidades (tablas), atributos/campos, relaciones entre los datos (tablas) y restricciones.



- **Nivel externo.** Se describen varios esquemas externos o vistas de usuario. Cada esquema externo describe la parte de la base de datos que interesa a un grupo de usuarios determinados y oculta a ese grupo el resto de la base de datos. En este nivel se puede utilizar un modelo conceptual o un modelo lógico para especificar los esquemas. Ese es el que percibe el usuario final mediante el uso de aplicaciones. Por ejemplo, cuando accedo a la página web de la liga de baloncesto estoy consultando una parte de los datos de sus bases de datos. Son lo que denominamos vistas.

Conviene recalcar que los tres niveles no son más que descripciones de los mismos datos, pero en distintos niveles de abstracción. Los únicos datos que existen realmente están a nivel físico, almacenados en un dispositivo, como puede ser un disco.

La arquitectura de tres niveles es útil para explicar el concepto de **independencia de datos**, que podemos definir como la capacidad para modificar el esquema en un nivel del sistema sin tener que modificar el esquema del nivel inmediato superior (por ejemplo, un cambio nivel externo no afectaría al nivel conceptual). Se pueden definir dos tipos de independencia de datos:

- **La independencia lógica.** Es la capacidad de modificar el esquema conceptual sin tener que alterar los esquemas externos ni los programas de aplicación. Se puede modificar el esquema conceptual para ampliar la base de datos o para reducirla. Si, por ejemplo, se reduce la base de datos eliminando una entidad, los esquemas externos que no se refieran a ella no deberán verse afectados.

- **La independencia física.** Es la capacidad de modificar el esquema interno sin tener que alterar el esquema conceptual (o los externos). Por ejemplo, puede ser necesario reorganizar ciertos ficheros físicos con el fin de mejorar el rendimiento de las operaciones de consulta o de actualización de datos. Dado que la independencia física se refiere solo a la separación entre las aplicaciones y las estructuras físicas de almacenamiento, es más fácil de conseguir que la independencia lógica.

3.2 MODELOS DE DATOS

La base de datos consiste entonces en los datos concretos referentes a un sistema o parte del mundo que hemos modelado (por ejemplo, nuestra base de datos de la liga de baloncesto incluye todos los datos relativos a jugadores, equipos, partidos, etc.).

Estos datos son sencillos de manejar cuando son unos pocos, pero cuando su volumen crece se requiere el uso de distintos modelos para facilitar el diseño de las mismas (si solo necesitamos registrar los nombres de jugadores y equipos no es necesario recurrir a ningún modelo).

Un **modelo de datos** es una colección de herramientas conceptuales para ***describir*** los datos, las relaciones que existen entre ellos y sus restricciones.

Un modelo nos proporciona mecanismos de abstracción para representar una parte del mundo cuyos datos nos interesan. Dicha representación, realizada en términos de un modelo dado, recibe el nombre de **esquema** y el conjunto de datos que representa es la **base de datos**.

3.3 TIPOS DE MODELOS

La arquitectura de tres niveles nos obliga a modelar nuestros datos en cada nivel. Nos centraremos en los dos primeros (global y externo), ya que los del nivel interno son específicos del software utilizado.

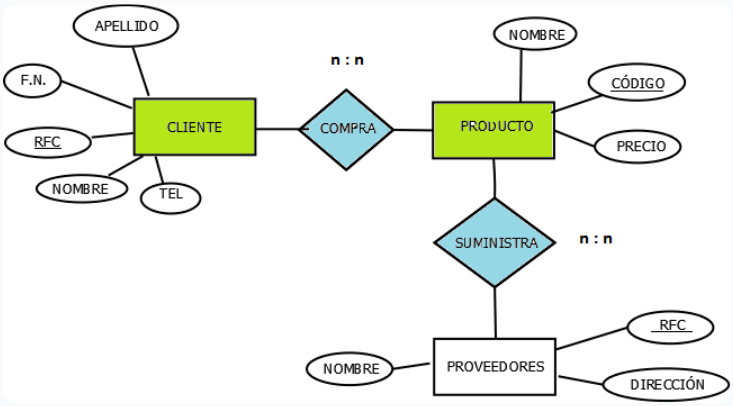
En este sentido distinguimos tres tipos de modelos:

- Modelos conceptuales.
- Modelos lógicos tradicionales.
- Modelos lógicos avanzados.

Modelos conceptuales

Se usan para describir datos a nivel global. Con este modelo representamos los datos de forma parecida a como nosotros los captamos en el mundo real. Este tipo de modelos tienen una capacidad de estructuración bastante flexible y permiten especificar restricciones de datos explícitamente. Existen diferentes modelos de este tipo, pero el más utilizado por su sencillez y eficiencia es el modelo Entidad-Relación.

Denominado por sus siglas MER, este modelo representa la realidad a través de entidades, que son objetos que existen y que se distinguen de otros por sus características. Por ejemplo, un jugador es una entidad con características como su altura, nombre, etc.



Estas características de las entidades en base de datos se llaman atributos. A su vez, una entidad se puede asociar o relacionar con más entidades a través de relaciones. Así un jugador está vinculado o relacionado con un equipo en virtud de la relación pertenece (jugador pertenece a equipo).

Este tipo de modelos utiliza una simbología para representar cada elemento. Lo estudiaremos más adelante.

Modelos lógicos tradicionales

Los tres modelos de datos más ampliamente aceptados son:

- **Modelo relacional.** En este modelo se representan los datos y las relaciones entre estos, a través de una colección de tablas, en las cuales las filas (tuplas) equivalen a

cada uno de los registros que contendrá la base de datos y las columnas corresponden a las características (atributos) de cada registro localizado en la tupla.



CLIENTE

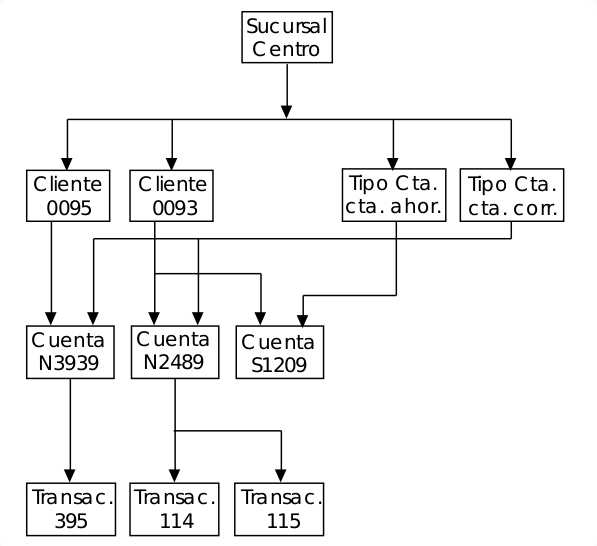
NOMBRE	CEDULA	CUENTA	CIUDAD
CANO	7.245.310	C-101	CALI
PEREZ	1.352.851	C-121	PASTO
TORO	9.874.115	C-203	BOGOTA
LOPEZ	9.765.398	C-302	BUGA
SERNA	2.458.698	C-109	TADO
VEGA	4.111.119	C-230	LIMA
CANO	7.245.310	C-209	CALI
PEREZ	1.352.851	C-209	PASTO

CUENTA

CUENTA	SALDO
C-101	50.000
C-121	120.000
C-203	70.000
C-302	98.000
C-209	42.000
C-109	108.500
C-230	59.000

También sirven para representar el nivel externo (vistas) de una base de datos.

- **Modelo de red** 📖**.** Éste es un modelo ligeramente distinto del jerárquico; su diferencia fundamental es la modificación del concepto de nodo: se permite que un mismo nodo tenga varios padres (posibilidad no permitida en el modelo jerárquico).

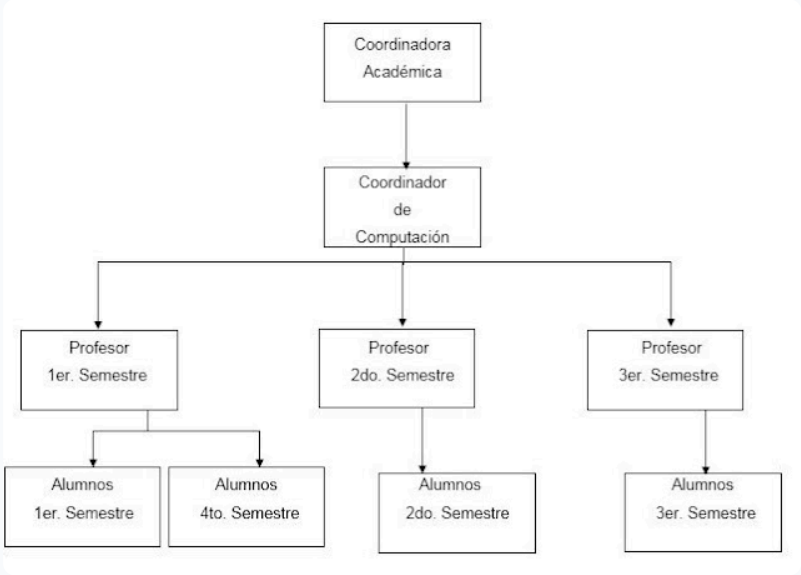


Fue una gran mejora con respecto al modelo jerárquico, ya que ofrecía una solución eficiente al problema de redundancia de datos; pero, aun así, la dificultad que significa administrar la información en una base de datos de red ha significado que sea un modelo utilizado en su mayoría por programadores más que por usuarios finales.

- **Modelo jerárquico.** 📖 Éstas son modelos de bases de datos que, como su nombre indica, almacenan su información en una estructura jerárquica. En este modelo los datos se organizan en una forma similar a un árbol (visto al revés), donde un nodo padre de información puede tener varios hijos. El nodo que no tiene padres es llamado raíz, y a los nodos que no tienen hijos se les conoce como nodos hoja.

Las bases de datos jerárquicas son especialmente útiles en el caso de aplicaciones que manejan un gran volumen de información y datos muy compartidos, permitiendo crear estructuras estables y de gran rendimiento a la hora de acceder a los mismos.

Es similar al modelo de red en cuanto a las relaciones y datos, ya que estos se representan por medio de registros de vínculos entre los mismos. La diferencia radica en que están organizados por gráficos de tipo árbol en lugar de gráficos arbitrarios.



Modelos lógicos avanzados 📖

Son modelos de datos relativamente recientes y cada vez más utilizados, sobre todo en aplicaciones específicas que manejan nuevos y más complejos tipos de datos.

- **Modelos de datos orientados a objetos.** Estos modelos son utilizados, sobre todo, en aplicaciones programadas bajo el paradigma de la orientación a objetos. Tratan de almacenar en la base de datos no solo los datos (estado), sino también la funcionalidad asociada (comportamiento). De este modo, una base de datos está formada por objetos relacionados entre sí, siendo los objetos entidades con un estado o datos asociados y un comportamiento o funcionalidad determinada.

- **Modelos de datos declarativos.** Estos modelos se dividen en deductivos y funcionales. Suelen usarse para bases de conocimiento, que no son más que bases de datos con mecanismos de consulta en los que el trabajo de extracción de información a partir de los datos recae en realidad sobre el sistema informático, en lugar de sobre el usuario. Estos mecanismos de consulta exigen que la información esté distribuida de manera que haga eficiente las búsquedas de los datos, ya que normalmente las consultas de este tipo requieren acceder una y otra vez a los datos en busca de patrones que se adecúen a las características de los datos que ha solicitado el usuario.

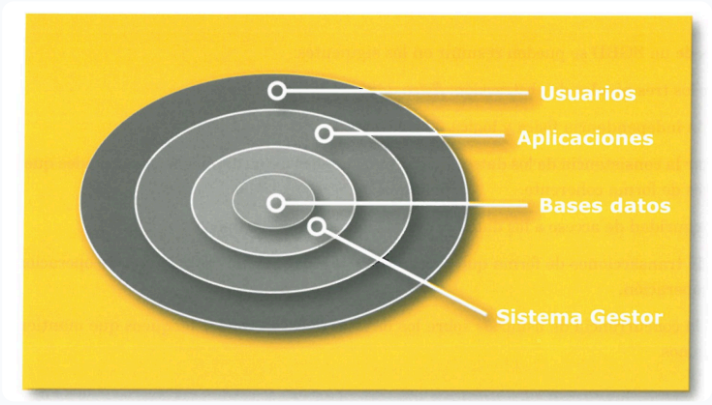
4 SISTEMAS GESTORES DE BASES DE DATOS

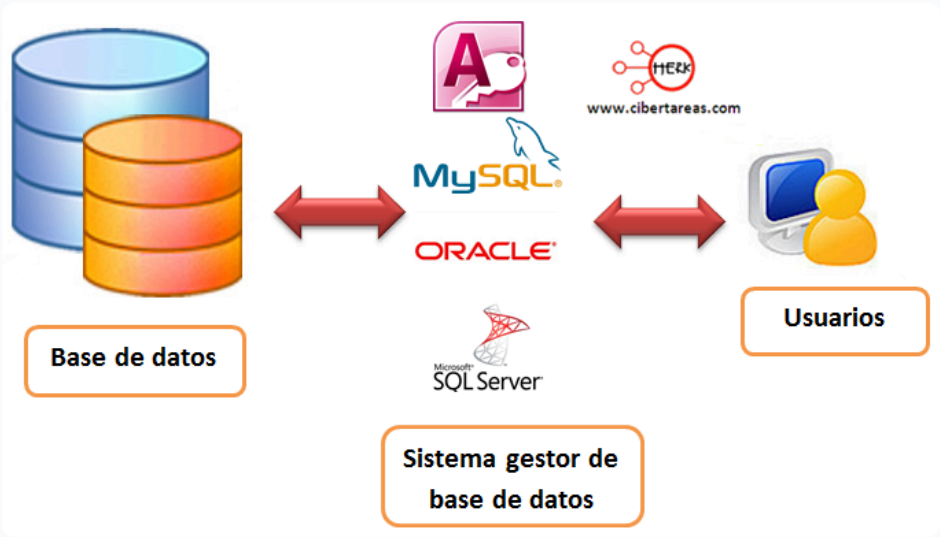
Los modelos nos permiten representar nuestra información de un modo sencillo y en un lenguaje común. Sin embargo, necesitamos un software que nos permita llevarlo a cabo o implementar dichos modelos. Este software es lo que denominaremos **SGBD**. Lo definiremos a continuación.

4.1 DEFINICIÓN Y OBJETIVOS

El **sistema de gestión de la base de datos (SGBD)** es una aplicación que permite a los usuarios definir, crear y mantener la base de datos y proporciona acceso controlado a la misma. Es una herramienta que sirve de interfaz entre el usuario y las bases de datos.

En el siguiente esquema se representa el funcionamiento de un sistema de información en el que los usuarios acceden a la información usando aplicaciones (por ejemplo, un formulario web) que, a su vez, se comunican con sistemas gestores, que son los que en última instancia acceden a los datos almacenados en las bases de datos mediante la interacción con el sistema operativo:





Se observa como en el nivel externo se encuentran los usuarios finales que tienen acceso a distintos esquemas, los cuales a su vez se derivan del esquema o representación lógica que, a su vez, se traduce a un esquema físico o forma de almacenamiento de los datos.

Los objetivos de un SGBD se pueden resumir en los siguientes:

- Asegurar los tres niveles de abstracción: físico, lógico y externo.
- Permitir la independencia física y lógica de los datos.
- Garantizar la consistencia de los datos, ya que puede haber datos duplicados o derivados que deben mantener sus valores de forma coherente.
- Ofrecer seguridad de acceso a los datos por parte de usuarios y grupos.
- Gestión de transacciones de forma que se garantice la ejecución de un conjunto de operaciones críticas como una sola operación.
- Permitir la concurrencia de usuarios sobre los mismos datos mediante bloqueos que mantienen la integridad de los mismos.

4.2 FUNCIONES DEL SISTEMA GESTOR DE BASE DE DATOS

Los SGBD del mercado cumplen con casi todas funciones que a continuación se enumeran:

1. Permiten a los usuarios almacenar datos, acceder a ellos y actualizarlos de forma sencilla y con un gran rendimiento, ocultando la complejidad y las características físicas de los dispositivos de almacenamiento.

2. Garantizan la integridad de los datos, respetando las reglas y restricciones que dicte el programador de la base de datos. Es decir, no permiten operaciones que dejen cierto conjunto de datos incompletos o incorrectos.

3. Integran, junto con el sistema operativo, un sistema de seguridad que garantiza el acceso a la información exclusivamente a aquellos usuarios que dispongan de autorización.

4. Proporcionan un diccionario de metadatos, que contiene el esquema de la base de datos, es decir, cómo están estructurados los datos en tablas, registros y campos, las relaciones entre los datos, usuarios, permisos, etc. Este diccionario de datos debe ser también accesible de la misma forma sencilla que es posible acceder al resto de datos.

5. Permiten el uso de transacciones, garantizan que todas las operaciones de la transacción se realicen correctamente, y en caso de alguna incidencia, deshacen los cambios sin ningún tipo de complicación adicional.

6. Ofrecen, mediante completas herramientas, estadísticas sobre el uso del gestor, registrando operaciones efectuadas, consultas solicitas, operaciones fallidas y cualquier tipo de incidencia. Es posible de este modo, monitorizar el uso de la base de datos, y permiten analizar hipotéticos malfuncionamientos.

7. Permiten la concurrencia, es decir, varios usuarios trabajando sobre un mismo conjunto de datos. Además, proporcionan mecanismos que permiten arbitrar operaciones conflictivas en el acceso o modificación de un dato al mismo tiempo por parte de varios usuarios.

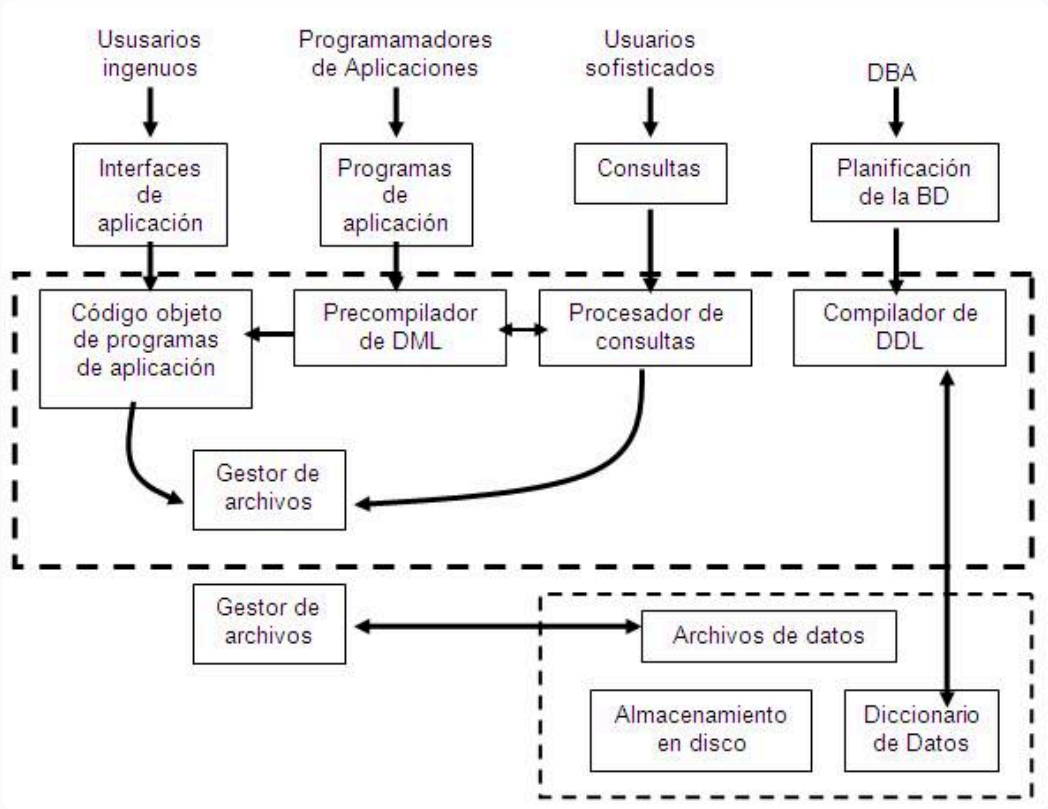
8. Independizan los datos de la aplicación o usuario que esté utilizándolos, haciendo más fácil su migración a otras plataformas.

9. Ofrecen conectividad con el exterior. De esta manera, se puede replicar y distribuir bases de datos. Además, todos los SGBD incorporan herramientas estándar de conectividad. El

protocolo ODBC está muy extendido como forma de comunicación entre bases de datos y aplicaciones externas.

10. Incorporan herramientas para la salvaguarda y restauración de la información en caso de desastre. Algunos gestores, tienen sofisticados mecanismos para poder establecer el estado de una base de datos en cualquier punto anterior en el tiempo. Además, deben ofrecer sencillas herramientas para la importación y exportación automática de la información.

4.3 COMPONENTES DE UN SGBD



Son los elementos que deben proporcionar los servicios comentados en la sección anterior. No se pueden generalizar ya que varían mucho según la tecnología. Sin embargo, normalmente todo SGBD incluye los siguientes:

Diccionario de datos

Esquemas que describen el contenido del SGBD incluyendo los distintos objetos con sus propiedades.

Objetos

- Tablas base y vistas (tablas derivadas).
- Consultas.
- Dominios y tipos definidos de datos.
- Restricciones de tabla y dominio y aserciones.

- Funciones y procedimientos almacenados.
- Disparadores o triggers.

Herramientas para...

- Seguridad: de modo que los usuarios no autorizados no puedan acceder a la base de datos.
- Integridad: que mantiene la integridad y la consistencia de los datos.
- El control de concurrencia: que permite el acceso compartido a la base de datos.
- El control de recuperación: que restablece la base de datos después de que se produzca un fallo del hardware o del software.
- Gestión del diccionario de datos (o catálogo): accesible por el usuario que contiene la descripción de los datos de la base de datos.
- Programación de aplicaciones.
- Importación/exportación de datos (migraciones).
- Distribución de datos.
- Replicación (arquitectura maestro-esclavo).
- Sincronización (de equipos replicados).

Optimizador de consultas

Para determinar la estrategia óptima para la ejecución de las consultas.

Gestión de transacciones

Este módulo realiza el procesamiento de las transacciones.

Planificador (scheduler)

Para programar y automatizar la realización de ciertas operaciones y procesos.

Copias de seguridad

Para garantizar que la base de datos se puede devolver a un estado consistente en caso de que se produzca algún fallo o error grave.

No todos los SGBD presentan la misma funcionalidad, depende de cada producto. En general, los grandes SGBD multiusuario ofrecen todas las funciones que se acaban de citar y muchas más. Los sistemas modernos son conjuntos de programas extremadamente complejos y sofisticados, con millones de líneas de código y con una documentación consistente en varios volúmenes. Los SGBD están en continua evolución, tratando de satisfacer los requerimientos de todo tipo de usuarios. Desde permitir el trabajo con nuevos objetos multimedia a sistemas de minería de datos capaces de detectar patrones de datos, pasando por sistemas de datos distribuidos entre múltiples equipos. Sin duda, a medida que pase el tiempo irán surgiendo nuevos requisitos que serán incorporados paulatinamente.

4.4 Lenguajes de datos

Son lenguajes para la manipulación de datos, tanto desde el punto de vista de su acceso y modificación como del control y seguridad de los mismos. Normalmente se distinguen tres tipos según su funcionalidad.

- **Lenguaje de definición de datos (DDL, Data Definition Language).** Sencillo lenguaje artificial para definir y describir los objetos de la base de datos, su estructura, relaciones y restricciones. Permite, entre otras cosas, la creación, eliminación y modificación de las estructuras de la base de datos, es decir, de la definición de las tablas, así como de índices y restricciones.

- **Lenguaje de control de datos (DCL, Data Control Language).** Encargado del control y seguridad de los datos (privilegios y modos de acceso, etc.). Este lenguaje permite especificar los permisos sobre los objetos de las bases de datos (tablas, vistas, procedimientos, etc.) así como la creación y eliminación de usuarios y cuentas.

- **Lenguaje de manipulación de datos (DML, Data Manipulation Language)** Es el lenguaje encargado de la manipulación del contenido de las bases de datos. Permite la inserción, actualización, eliminación y consulta de datos en las tablas de las bases de datos.

Para todos estos lenguajes se usa principalmente el lenguaje SQL (Structured Query Language). Incluye instrucciones para los tres tipos de lenguajes comentados y por su sencillez y potencia se ha convertido en el lenguaje estándar de los SGBD relacionales.

4.5 USUARIOS DE LOS SGBD

Generalmente, distinguimos cuatro grupos de usuarios de sistemas gestores de bases de datos: los usuarios administradores, los diseñadores de la base de datos, los programadores y los usuarios de aplicaciones que interactúan con las bases de datos.

Administradores

Trabajan en el nivel de abstracción físico relacionado con el almacenamiento.

Distinguimos a los administradores del propio sistema gestor encargados de la instalación y configuración del sistema, del control de acceso a los recursos, de la seguridad y de la monitorización y optimización del sistema gestor.

Por su parte, los administradores de bases de datos se encargan del diseño físico de la misma, implementación y mantenimiento de la base de datos.

Diseñadores de la base de datos

Realizan el diseño lógico de la base de datos, debiendo identificar los datos, las relaciones entre datos y las restricciones sobre los datos y sus relaciones. El diseñador de la base de datos debe tener un profundo conocimiento de los datos de la empresa y también debe conocer sus reglas de negocio. Las reglas de negocio describen las características principales de los datos tal y como los ve la empresa. Para obtener un buen resultado, el diseñador de la base de datos debe implicar en el desarrollo del modelo de datos a todos los usuarios de la base de datos, tan pronto como sea posible. El diseño lógico de la base de datos es

independiente del SGBD concreto que se vaya a utilizar, es independiente de los programas de aplicación, de los lenguajes de programación y de cualquier otra consideración física.

Programadores

Tanto de aplicaciones que, mediante API de lenguajes de programación interactúan con las bases de datos como de objetos de la base de datos, como rutinas almacenadas o disparadores. Estas aplicaciones servirán a los usuarios finales para, de una forma amigable, poder consultar datos, insertarlos, actualizarlos y eliminarlos.

Usuarios finales

Trabajan en el nivel externo mediante vistas o porciones de las bases de datos. Son clientes de las bases de datos que hacen uso de ellas sin conocer en absoluto su funcionamiento y organización interna. Son personas con pocos o nulos conocimientos de informática.

4.6 TIPOS DE SGBD

Existen numerosos SGBD en el mercado que podemos clasificar según los siguientes criterios:

Modelo lógico en el que se basan

- Jerárquico.
- En red.
- Relacional.
- Objeto-relacional.
- Orientado a objetos.

Número de usuarios

- Monousuario: solo permiten un usuario.
- Multiusuario: permiten la conexión de varios usuarios.

Número de sitios

- Centralizados: en un solo servidor o equipo.
- Distribuidos: en varios equipos que pueden ser homogéneos y heterogéneos.

Ámbito de aplicación

- Propósito General: orientados a toda clase de aplicaciones.
- Propósito Específico: centradas en un tipo específico de aplicaciones.

Tipos de datos

- Sistemas relacionales estándar: manejan tipos básicos (int, char, etc.).
- XML: para el caso de bases de datos que trabajan con documentos xml.

- Objeto-relacionales: para bases relacionales que incorporan tipos complejos de datos.
- De objetos: para bases de datos que soportan tipos de objeto con datos y métodos asociados.

Lenguajes soportados

- SQL estándar.
- NoSQL o nuevo lenguaje de consulta: menos estructurado y orientado a bases documentales o de tipo clave-valor. Es muy útil para manejar consultas de grandes cantidades de datos distribuidos en clusters de servidores.

📖 Dentro de todos ellos los que con gran diferencia se han impuesto en casi todos los ámbitos del desarrollo han sido los basados en el modelo relacional. Ello se ha debido principalmente a su flexibilidad y sencillez de manejo. Igualmente, conviene destacar la amplia implantación del lenguaje SQL, que se ha convertido en un estándar para el manejo de datos en el modelo relacional, lo que ha supuesto una ventaja adicional para su desarrollo ya que, además, ha incorporado (en su versión SQL1999) aspectos importantes de la orientación a objetos.

Cabe destacar, sin embargo, la creciente popularidad de otros lenguajes como NoSQL (Not Only SQL), que se han desarrollado para adaptarse a datos masivos y dispersos en muchos servidores.

Usos de las bases de datos 📖

Las bases de datos son ubicuas, están en cualquier tipo de sistema informático, a continuación se exponen sólo algunos ejemplos de sus usos más frecuentes:

- Bases de datos Administrativas: Cualquier empresa necesita registrar y relacionar sus clientes, pedidos, facturas, productos, etc.
- Bases de datos Contables: También es necesario gestionar los pagos, balances de pérdidas y ganancias, patrimonio, declaraciones de hacienda…
- Bases de datos para motores de búsquedas: Por ejemplo Google o Altavista, tienen una base de datos gigantesca donde almacenan información sobre todos los documentos de Internet. Posteriormente millones de usuarios buscan en la BD de estos motores.
- Científicas: Recolección de datos climáticos y medioambientales, químicos, geológicos...
- Configuraciones: Almacenan datos de configuración de un sistema informático, como por ejemplo, el registro de windows.
- Bibliotecas: Almacenan información bibliográfica, por ejemplo, la famosa tienda virtual amazon o la biblioteca de un instituto.
- Censos: Guardan información demográfica de pueblos, ciudades y países.
- Virus: Los antivirus guardan información sobre todos los potenciales software maliciosos.
- Otros muchos usos: Militares, videojuegos, deportes, etc.

4.7 SISTEMAS GESTORES DE BD COMERCIALES Y LIBRES 📖

Con el advenimiento de Internet, el software libre se ha consolidado como alternativa, técnicamente viable y económicamente sostenible al software comercial, contrariamente a lo que a menudo se piensa, convirtiéndo el software libre como otra alternativa para ofrecer los mismos servicios a un coste cada vez más reducido.

Sin embargo, debe notarse que lo que comúnmente se denomina software libre no significa que sea gratuito ya que muchas empresas ofrecen servicios de soporte y mantenimiento para productos (en ocasiones de pago como Red Hat o Suse) pero cuyo código sigue siendo accesible.

Estas alternativas se encuentran tanto para herramientas de ofimática como Libreoffice, Openoffice frente a Microsoft Office, como herramientas mucho más avanzadas y de de propósito general, como MySQL frente SQL Server o a un nivel superior en cuanto a potencialidad, como PostgreSQL frente a Oracle, entre otros.

No obstante, cada vez más los fabricantes ofrecen versiones gratuitas (que no libres), aunque con limitaciones en su funcionalidad, de sus productos que permiten probarlos y aprender sus interioridades. Estas versiones se suelen denominar de tipo express.

Con independencia de la versión de producto también disponemos, tanto en los productos de software libre como en los de pago, de cada vez más documentación en línea que facilita enormemente su aprendizaje aunque en este sentido los productos de software libre destacan ya que los usuarios y comunidades generan una gran cantidad de documentación que está permanentemente actualizada.

Otro factor importante es el humano. Usar software libre a menudo implica un mayor conocimiento del producto y, por tanto, personal más cualificado. Por el contrario, esto permite un mayor control sobre el mismo y sobre las aplicaciones a desarrollar, además de una gran independencia con respecto al proveedor del mismo, por no hablar del ahorro en cuanto a licencias y costes de mantenimiento y soporte. Los sistemas comerciales, por el contrario, suelen ser más cerrados y rígidos. En todo caso todas las tecnologías nombradas disponen de servicios de soporte y documentación de gran calidad.

En definitiva, usar software libre o no es una elección del consumidor. Debe considerar estos y otros factores de la manera que mejor se adapte a sus necesidades.

No se trata tanto de defender "qué es mejor en sí mismo", sino de tener la información y poder elegir "qué es mejor según mis circunstancias".

A la hora de decidirse entre un sistema libre o comercial, un factor importante es si disponemos de personal cualificado, en cuyo caso un sistema libre es más barato y potente y nos dará más posibilidades y flexibilidad.

ACTIVIDADES 1.2

1. ¿Qué se entiende por diseño físico de una base de datos? ¿Qué usuarios son los responsables del mismo?

2. Averigüe en qué consiste y para qué sirve la minería de datos.

3. ¿Qué lenguaje específico usa SQL Server para implementar el lenguaje SQL?

4. Busque al menos 2 sistemas gestores libres (Open Source) y 2 comerciales e investigue sus ventajas e inconvenientes.

5. Haga una investigación sobre las diferencias fundamentales entre los SGBD orientados a modelos relacionales de datos y los basados en NoSQL, como CouchDB, Redis o Cassandra.

6. Buscar los cinco SGBD más utilizados (unos datos que estén actualizados). Incluir sus características principales.

7. Crea una tabla como las que aparecen en este documento(de alumnos, clientes, coches, equipos de fútbol, cantantes,...). Debe de tener 4 o 5 registros (filas) y 4 o 5 campos (columnas) como mínimo. Indicar cual es la clave, nombre de los campos, tipos de datos (entero, decimal, texto, fecha, hora, booleano, otro).

8. Busca en Internet las leyes de Codd para el funcionamiento de sistemas gestores de bases de datos relacionales

9. Indica las funciones que proporcionan los SGBD actuales.

10. Con los apuntes proporcionados buscar los siguientes conceptos y completar con la definición correspondiente: Fichero, Base de datos, Sistema gestor de bases de datos, Dato, Campo, registro, tabla, consulta, vista y esquema

5 BD CENTRALIZADAS Y DISTRIBUIDAS

En un sistema de bases de datos centralizado todos los componentes (software, datos y soportes físicos) residen en un único lugar físico. Los clientes (aplicaciones, funciones, programas cliente, usuarios) acceden al sistema a través de distintas interfaces que se conectan al servidor. Sin embargo, existe otra arquitectura cada vez más extendida en la que se opta por un esquema distribuido en el que los componentes se distribuyen en distintos computadores comunicados a través de una red de cómputo. Dichos sistemas se conocen con el nombre de Sistemas Gestores de Bases de Datos Distribuidas o DDMGS.

Una **base de datos distribuida** es una colección de datos que pertenece lógicamente al mismo sistema pero que se encuentra físicamente almacenada en distintas máquinas conectadas por una red.

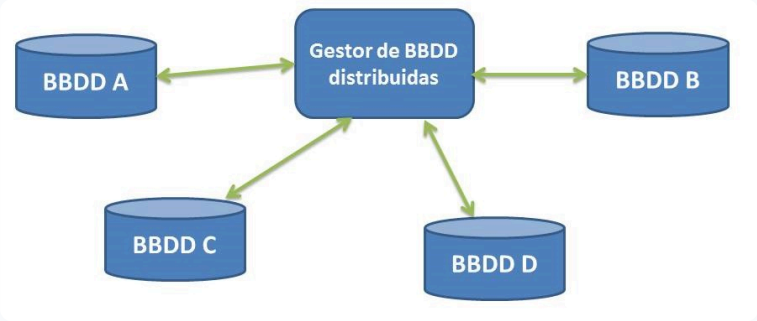
El surgimiento de las mismas se debe a varias razones entre las que destacan las siguientes:

- **Mejora de rendimiento.** Cuando grandes bases de datos se distribuyen en varios sitios las consultas y transacciones que afectan a un solo sitio son más rápidas al ser más pequeña la base de datos local. Los sitios no se ven congestionados por muchas transacciones, ya que éstas se dispersan entre varios sistemas. De este modo, cuando una transacción requiera acceso a más de un sitio, estos pueden hacerse en paralelo, de forma que se reduce el tiempo de respuesta.

- **Fiabilidad.** Definida como la probabilidad de que un sistema esté activo en un tiempo dado. Al distribuirse los datos es más fácil asegurar la fiabilidad del

sistema, ya que si falla algún sitio es poco probable que afecte a la mayoría de transacciones.

- **Disponibilidad.** Es la probabilidad de que un sistema esté activo de manera continuada durante un tiempo. Se ve mejorada por las mismas razones expuestas anteriormente. Esta característica se ve mejorada especialmente por la replicación de datos en varios sistemas.



Tipos de aplicaciones

Muchas aplicaciones tienen un marcado carácter distribuido al estar sus componentes dispersos en distintos sitios. Por ejemplo, una cadena de supermercados puede tener distintas sedes en distintas ciudades de forma que los clientes puedan acceder solamente a sitios y datos de su ciudad.

Por el contrario, distribuir implica una mayor complejidad en el diseño, implementación y gestión de los datos, para lo cual un buen SGBDD debe incorporar, además de los componentes habituales en un SGBD centralizado:

- Tener acceso a sitios remotos e intercambiar información con los mismos para la ejecución de consultas y transacciones.
- Disponer de un catálogo con información sobre cómo están distribuidos y replicados los datos del sistema distribuido.
- Poder optimizar consultas y transacciones sobre datos que estén en más de un sitio.
- Mantener la integridad en los permisos sobre datos replicados.
- Mantener la consistencia de las copias de un elemento replicado.
- Garantizar la recuperación del sistema en una caída.

Todo ello eleva enormemente la complejidad de tal SGBDD. Debemos añadir además la dificultad en el diseño óptimo de bases de datos distribuidas, especialmente en cuanto a las dos decisiones importantes: qué datos distribuir y qué datos replicar.

5.1 TÉCNICAS DE FRAGMENTACIÓN, REPLICACIÓN Y DISTRIBUCIÓN

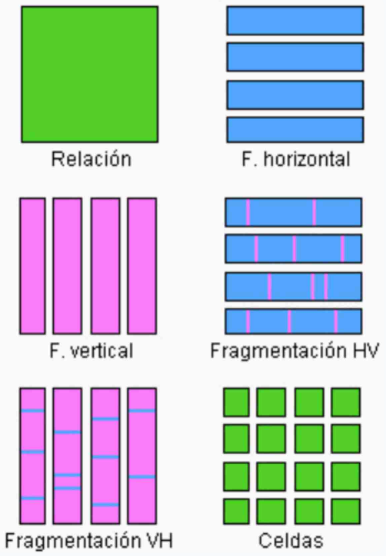
Diseñar bases de datos distribuidas implica decidir sobre cómo fragmentarlas, cómo replicarlas y cómo distribuirlas, además del diseño previo de la base de datos en sí.

La información sobre cómo se hace este proceso se almacena en el catálogo global del sistema al que tiene acceso el cliente cuando quiere acceder a la información.

Veremos ahora distintas técnicas para hacer lo anterior.

Fragmentación

Consideraremos a nuestra base de datos formada por unidades lógicas que son las relaciones o tablas. Supongamos que en nuestro ejemplo de banca electrónica queremos separar a nuestros clientes por regiones de España para facilitar el acceso. Entonces necesitaríamos dividir la tabla de clientes en tres partes almacenando cada una en un computador, es lo que denominamos fragmentación horizontal que divide las tuplas según una o varias condiciones sobre distintos atributos, en este caso sobre el campo región en la tabla clientes.



En otra situación podríamos querer dividir la información de un cliente en sus campos más accedidos y ligeros (nombre, apellidos, dirección) y almacenarla en un servidor más potente, mientras que otros campos más pesados (currículum, foto, historial, etc.) podrían almacenarse en otro servidor con menos capacidad de proceso pero más almacenamiento. Es lo que denominamos fragmentación vertical y debe hacerse con cuidado, ya que tenemos que incluir un atributo común, usualmente una clave, en ambos fragmentos para poder recuperar la información completa cuando sea necesario.

Por supuesto, existe también la posibilidad de una fragmentación mixta que incluya los dos casos comentados.

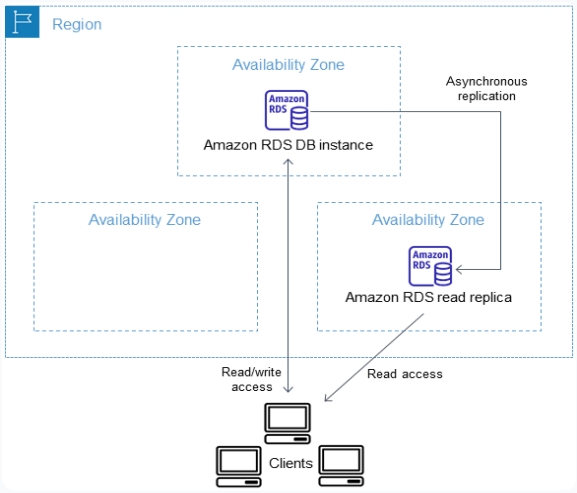
Debemos ser conscientes de que fragmentar es un proceso complejo que solo debe hacerse cuando se considere necesario por motivos de eficiencia, ya que al hacerlo hacemos también más complejo (más costoso en términos de CPU y consumo de recursos en general) el acceso a los datos por parte de los clientes.

La información sobre la fragmentación se guarda en lo que se denomina un esquema de fragmentación, que es una definición de todos los atributos de la base de datos y cómo están fragmentados. Un esquema de reparto permite definir dónde repartiremos los fragmentos. Si se da el caso de repartirlos en más de un sitio diremos que está replicado.

Fragmentar permite acercar los datos al consumidor, reducir el tráfico de red y mejorar la disponibilidad de los datos.

Replicación

Es fundamentalmente útil para mejorar la disponibilidad de los datos. El caso más extremo es la replicación en todos los sitios de la misma copia de la base de datos, que también es el caso que ofrece mayor disponibilidad, aunque puede reducir considerablemente la eficiencia en operaciones de actualización dado que cada operación debe realizarse en cada base de datos y esto puede traer problemas de integridad y consistencia de los mismos. La replicación, como hemos señalado, distribuye la carga, acelera consultas y mejora la disponibilidad pero, sobre todo, es óptima para sistemas con muchas lecturas. Sin embargo, requiere muchas más actualizaciones con el consiguiente consumo de almacenamiento.



ACTIVIDADES 1.3

1. Investigue sobre SGBD, comerciales o libres, que soporten replicación y fragmentación de datos.

2. ¿Cuál es el principal problema que tiene la replicación de datos?

3. Analice la conveniencia de replicar y fragmentar datos en los siguientes sistemas: una base de datos de películas on line y una base de datos de un banco.

4. Explica qué son los **metadatos** y qué información específica almacena el **diccionario de datos (o catálogo)** en un sistema gestor de bases de datos.

5. Pon tres ejemplos de información administrativa o estructural que guarde el catálogo (por ejemplo: definiciones de tablas, usuarios/permisos, restricciones, etc.) y explica por qué es fundamental para el funcionamiento automático del propio SGBD.

6. Describe los tres niveles de la arquitectura ANSI/SPARC (**nivel interno o físico**, **nivel conceptual o global**, y **nivel externo o vistas**). Indica qué representa cada uno y qué tipo de usuario interactúa principalmente en dicho nivel.

7. Explica la diferencia entre **independencia lógica** e **independencia física** de los datos. Pon un ejemplo práctico de un cambio que afecte al nivel físico sin alterar el conceptual, y un cambio en el conceptual que no afecte a los esquemas externos existentes.

8. Elabora una tabla comparativa donde se analicen las ventajas y desventajas de utilizar un **SGBD frente al almacenamiento tradicional en sistemas de ficheros**, teniendo en cuenta factores clave como: *redundancia de datos, consistencia, control de concurrencia, seguridad/autorizaciones e independencia de datos*.